



TEMA II

DESARROLLO DE APLICACIONES BACK EN PHP

*"Cuanto más trabajo,
más suerte parezco tener"*
Thomas Jefferson

9. PATRONES DE DISEÑO

Un patrón de diseño es una solución a un problema típico que nos encontraremos de forma reiterada al codificar aplicaciones. Nuestras aplicaciones tienen partes en común con otras: acceso a bbdd, creación de objetos..

Un patrón de diseño son "ciertas reglas" que seguiremos a la hora de implementar dichos problemas. No son "obligatorias" pero suelen ser "la mejor manera de diseñar y organizar mi código". Los patrones ayudan a estandarizar código, a que sea más legible y su diseño más comprensible para otros programadores.

9.1 Persistencia

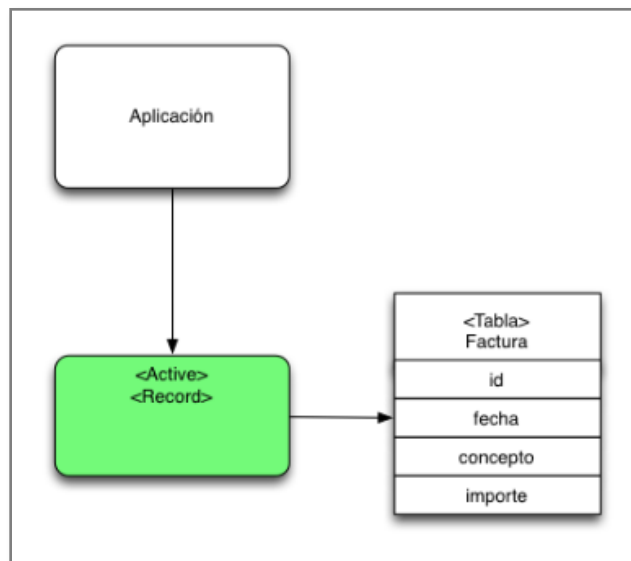
"En programación, la persistencia es la acción de preservar la información de un objeto de forma permanente (guardarlo), pero a su vez también se refiere a poder recuperar la información del mismo (leerlo) para que pueda ser nuevamente utilizado"

Patrones de persistencia

Patrones de diseño cuyo objetivo es almacenar objetos en un repositorio, siendo este una base de datos, un archivo, memoria RAM...

En lo que se refiere a bbdd, los patrones más utilizados son: Active record, DAO, DTO,...

- **Active record:** Un objeto que encapsula el acceso a base de datos y la manipulación de los datos de la misma. Para cada tabla se diseña una clase con los métodos de acceso a la misma

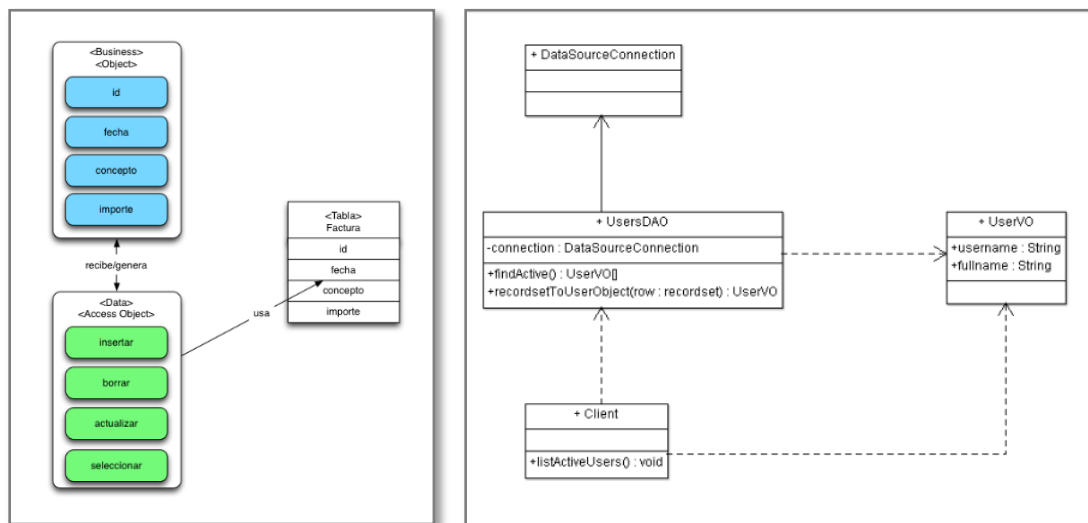


Si a este patrón le añadimos un motor de persistencia, por ejemplo en php

```
$cars = Car::all(); // nos devuelve un array de objetos con todos los coches
$car = Car::find(1); // nos devuelve el coche con id=1
$driver_name = $car->driver->name; // nos devuelve el nombre del conductor
$car->price = 10000; // actualizamos el precio del coche.
$car->save(); // Hacemos los cambios persistentes.
$car->delete(); // Borramos el coche anterior.
$scheapest_cars = Car::all(array('conditions' => 'price < 10000'));
```

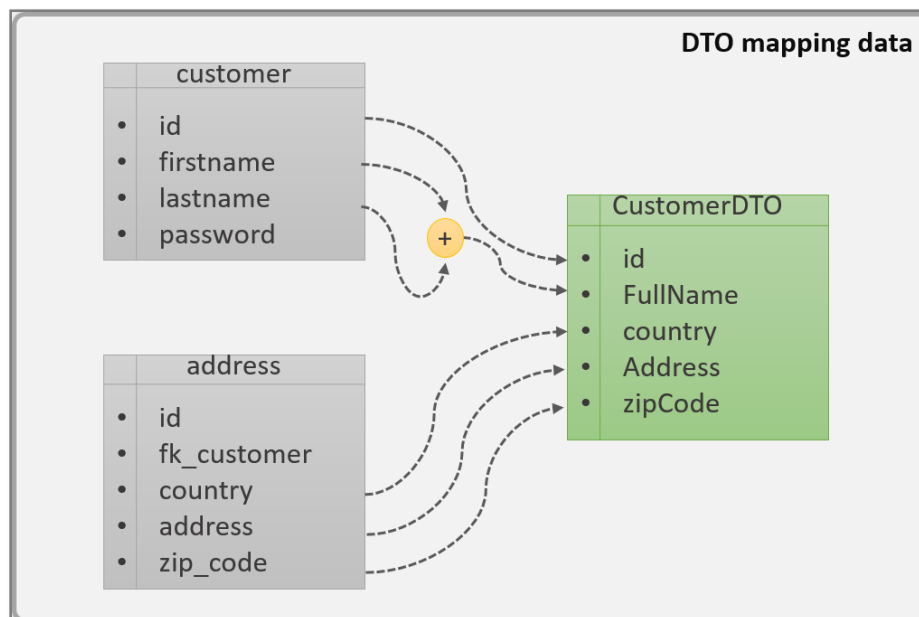
- **DAO(Data Access Object):** este patrón separa aun más las tareas, tendremos clases para almacenar los valores (persistencia, lo que nosotros llamamos modelos) y clases para el código que accede a las tablas. A los primeros objetos se les denomina (Value Object ó Transfer Object VO) que no es más que una clase donde cada atributo se corresponde con una columna de nuestra tabla (en el más simple de los casos). A los segundos les llamamos DAO, que no es más que una clase con los métodos de acceso a una tabla.

En un sentido estricto, según este patrón tenemos dos clases por cada tabla de la bbdd.



- **Data Transfer Object(DTO):** uno de los problemas más comunes cuando desarrollamos aplicaciones, es diseñar la forma en que la información debe llegar a la capa de presentación, muchas veces por "desconocimiento o pereza", utilizamos las clases de entidades(VO) para retornar los datos, lo que significa que retornamos más datos de los necesarios o incluso, que vamos en más de una ocasión al servidor a recuperar datos de varias tablas.

El patrón DTO tiene como finalidad crear un objeto, con una serie de atributos que puedan ser enviados o recuperados del servidor en **una sola invocación**, de tal forma que un DTO puede contener información de **múltiples tablas** y concentrarlas en una única clase simple.



Podemos, y a veces debemos, omitir información que el usuario no requiere (password).

Si bien un DTO es simplemente un objeto plano, sí que tiene que cumplir algunas reglas para poder considerar que hemos creado un DTO correctamente implementado:

- **Solo lectura:** Dado que el objetivo de un DTO es utilizarlo como un objeto de transferencia entre el cliente y el servidor, es importante evitar tener operaciones de negocio o métodos que realicen cálculos sobre los datos, solo debemos tener los métodos GET y SET de los respectivos atributos del DTO
- **Serializable:** si los objetos van a viajar por la red, deberán ser serializables, no sólo la clase en sí, también todos los atributos que contenga el DTO. Un error clásico en Java es, por ejemplo, crear atributos de tipo Date o Calendar para transmitir la fecha u hora, ya que estos no tienen una forma estándar para serializarse por ejemplo en Webservices o REST.